Spring Security Theory - Residentia Project

Complete Interview Guide

🎯 What is Spring Security?

Spring Security is a **powerful authentication and authorization framework** for Java applications. It intercepts HTTP requests **before they reach your controllers** and enforces security rules.

Core Functions:

1. **Authentication** - "Who are you?" (verify user identity)
2. **Authorization** - "What can you do?" (check permissions)
3. **Protection** - CSRF, CORS, session security

🔄 How Spring Security Works - The Filter Chain

Request Flow Through Security Filters

```
Client Browser
    │
    ├── HTTP Request with JWT Token
    │   (Authorization: Bearer eyJhbGc...)
    │
    ▼
┌──────────────────────────────────────────────────────────┐
│         Servlet Container (Tomcat)                       │
│                                                          │
│  ┌──────────────────────────────────────────────────┐   │
│  │    DelegatingFilterProxy                         │   │
│  │    (Bridge between Servlet & Spring)             │   │
│  └──────────────────┬───────────────────────────────┘   │
│                     │                                    │
│                     ▼                                    │
│  ┌──────────────────────────────────────────────────┐   │
│  │     FilterChainProxy (Spring Security)           │   │
│  │                                                  │   │
│  │  Filter 1: CORS Filter               ✓           │   │
│  │  Filter 2: CSRF Filter               ✗ (OFF)     │   │
│  │  Filter 3: JwtAuthenticationFilter   ✓ CUSTOM    │   │
│  │  Filter 4: ExceptionTranslationFilter✓ 401/403   │   │
│  │  Filter 5: AuthorizationFilter       ✓ Roles     │   │
│  └──────────────────┬───────────────────────────────┘   │
└─────────────────────┼────────────────────────────────────┘
                      │
                      ▼
            DispatcherServlet
                      │
```

```
                    ▼
        @RestController Methods
           (e.g., /api/owner/properties)
```

Key Points:

- Filters execute in a **specific order** (can't be changed randomly)
- Each filter can **allow** or **block** the request
- Our custom `JwtAuthenticationFilter` runs **BEFORE** authorization checks

🔐 Authentication Flow in Residentia Project

Step 1: User Login (Initial Authentication)

```
User → /api/auth/login
       {
         "email": "owner@example.com",
         "password": "password123",
         "role": "OWNER"
       }
       │
       ▼
┌──────────────────────────────────────────┐
│  AuthController.login()                  │
│                                          │
│  1. Find user by email (check DB)        │
│  2. Verify password (BCrypt)             │
│  3. Check if account is active           │
│  4. Generate JWT token                   │
└──────────────────────────────────────────┘
               │
               │
               ▼
        JWT Token Generated
     (Contains: userId, email, role)
               │
               ▼
        Response to Client
     {
       "token": "eyJhbGciOiJIUzI1NiIs...",
       "email": "owner@example.com",
       "name": "John Doe",
       "role": "OWNER"
     }
               │
               ▼
     Stored in localStorage/sessionStorage
     (Frontend: axios interceptor adds to every request)
```

Step 2: Subsequent Requests (Token Validation)

```
Client → Any Protected API
         Headers: {
           Authorization: "Bearer eyJhbGciOiJIUzI1NiIs..."
         }
         │
         ▼
┌──────────────────────────────────────────────────────┐
│  JwtAuthenticationFilter.doFilterInternal()          │
│                                                      │
│  1. Extract JWT from "Authorization" header          │
│  2. Validate token signature & expiration            │
│  3. Extract claims (email, role, userId)             │
│  4. Create Authentication object                     │
│  5. Store in SecurityContextHolder                   │
│  6. Continue to next filter                          │
└──────────────────────────────────────────────────────┘
              │
              │
              ▼
┌──────────────────────────────────────────────────────┐
│  AuthorizationFilter                                 │
│                                                      │
│  Checks if user has required role:                   │
│  - /api/owner/** → Requires authentication           │
│  - /api/admin/** → Requires authentication           │
│  - /api/client/** → Requires authentication          │
│                                                      │
│  If authorized → ✓ Allow                            │
│  If not → ✗ 403 Forbidden                           │
└──────────────────────────────────────────────────────┘
              │
              │
              ▼
        Controller Method Executes
```

🔧 Key Components in Our Project

1. SecurityConfig.java

Purpose: Main configuration class for Spring Security

Key Configurations:

```java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    // ☑ 1. Password Encoder (BCrypt)
    @Bean
```

```
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    // ☑ 2. JWT Filter Registration
    @Bean
    public JwtAuthenticationFilter jwtAuthenticationFilter() {
        return new JwtAuthenticationFilter(jwtTokenProvider);
    }

    // ☑ 3. Security Rules
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) {
        http
            .csrf(csrf -> csrf.disable())  // ✕ CSRF disabled (JWT handles
security)
            .sessionManagement(session ->
                session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)  //
✕ No sessions
            )
            .authorizeHttpRequests(authz -> authz
                // PUBLIC endpoints (no authentication needed)
                .requestMatchers("/api/auth/login").permitAll()
                .requestMatchers("/api/auth/register/**").permitAll()
                .requestMatchers(HttpMethod.GET,
"/api/client/properties").permitAll()

                // PROTECTED endpoints (authentication required)
                .requestMatchers("/api/owner/**").authenticated()
                .requestMatchers("/api/admin/**").authenticated()
                .requestMatchers("/api/client/**").authenticated()
            )
            .addFilterBefore(jwtAuthenticationFilter(),
                             UsernamePasswordAuthenticationFilter.class);
    }
}
```

Interview Q&A:

Q: Why is CSRF disabled? A: Because we use **JWT (stateless authentication)**. CSRF attacks target session-based authentication. Since we don't use cookies/sessions, CSRF protection is unnecessary.

Q: What is SessionCreationPolicy.STATELESS? A: It tells Spring Security **NOT to create or use HTTP sessions**. Every request must include a JWT token. This makes the API **scalable and stateless**.

---

2. JwtAuthenticationFilter.java

Purpose: Custom filter that validates JWT token on every request

Key Actions:

```java
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) {

        // Step 1: Extract JWT from header
        String jwt = getJwtFromRequest(request);  // "Bearer token..."

        if (jwt != null && jwtTokenProvider.validateToken(jwt)) {

            // Step 2: Extract user info from token
            String email = jwtTokenProvider.getEmailFromToken(jwt);
            String role = jwtTokenProvider.getRoleFromToken(jwt);
            Long userId = jwtTokenProvider.getOwnerIdFromToken(jwt);

            // Step 3: Create Authentication object
            List<GrantedAuthority> authorities =
                List.of(new SimpleGrantedAuthority("ROLE_" + role));

            UsernamePasswordAuthenticationToken auth =
                new UsernamePasswordAuthenticationToken(email, null, authorities);

            // Step 4: Set authentication in SecurityContext
            SecurityContextHolder.getContext().setAuthentication(auth);

            // Step 5: Store user info in request attributes
            request.setAttribute("email", email);
            if ("OWNER".equals(role)) {
                request.setAttribute("ownerId", userId);
            }
        }

        // Step 6: Continue to next filter
        filterChain.doFilter(request, response);
    }
}
```

Interview Q&A:

Q: Why extend OncePerRequestFilter? A: It guarantees the filter executes **exactly once per request**, even in forwarded/dispatched requests.

Q: Where is the authentication stored? A: In `SecurityContextHolder`, which is a **ThreadLocal** storage. Each request thread has its own security context.

Q: How do controllers access the authenticated user? A: Through `request.getAttribute("email")` or by using `@AuthenticationPrincipal` annotation.

---

3. JwtTokenProvider.java

Purpose: Generate and validate JWT tokens

Key Methods:

```
public class JwtTokenProvider {

    @Value("${jwt.secret}")
    private String jwtSecret;  // Secret key for signing

    @Value("${jwt.expiration}")
    private long jwtExpiration;  // Token validity (e.g., 24 hours)

    // Generate token when user logs in
    public String generateToken(Long userId, String email, String role) {
        Date now = new Date();
        Date expiryDate = new Date(now.getTime() + jwtExpiration);

        return Jwts.builder()
            .setSubject(email)
            .claim("userId", userId)
            .claim("role", role)
            .setIssuedAt(now)
            .setExpiration(expiryDate)
            .signWith(SignatureAlgorithm.HS512, jwtSecret)
            .compact();
    }

    // Validate token signature and expiration
    public boolean validateToken(String token) {
        try {
            Jwts.parser()
                .setSigningKey(jwtSecret)
                .parseClaimsJws(token);
            return true;
        } catch (ExpiredJwtException ex) {
            log.error("JWT token expired");
            return false;
        } catch (Exception ex) {
            log.error("Invalid JWT token");
            return false;
        }
    }

    // Extract email from token
    public String getEmailFromToken(String token) {
        Claims claims = Jwts.parser()
            .setSigningKey(jwtSecret)
            .parseClaimsJws(token)
            .getBody();
        return claims.getSubject();
    }
}
```

🛡️ Authorization (Role-Based Access Control)

How Roles Work in Our Project

Role	Access	Endpoints
CLIENT	Browse properties, make bookings, write reviews	`/api/client/**`
OWNER	Manage properties, view bookings, accept/reject	`/api/owner/**`
ADMIN	Manage all users, properties, bookings	`/api/admin/**`

Authorization Flow

```
Request → JwtAuthenticationFilter (validates token)
          │
          ├── Extracts role from JWT
          │   (e.g., "ROLE_OWNER")
          │
          ▼
       SecurityContextHolder stores role
          │
          ▼
     AuthorizationFilter checks
          │
          ├── Endpoint: /api/owner/properties
          ├── Required: authenticated()
          ├── User Role: ROLE_OWNER
          │
          ▼
     ☑ Access GRANTED
          │
          ▼
     Controller executes
```

🌐 CORS Configuration

Purpose: Allow frontend (React) to call backend APIs from different origins

```java
@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration config = new CorsConfiguration();

    // Allow localhost ports (dev environment)
    config.setAllowedOriginPatterns(Arrays.asList(
        "http://localhost:*",
        "http://localhost:5173",  // Vite dev server
```

```
        "http://localhost:3000"   // React dev server
    ));

    // Allow all HTTP methods
    config.setAllowedMethods(Arrays.asList("GET", "POST", "PUT", "DELETE",
"OPTIONS"));

    // Allow Authorization header (for JWT)
    config.setAllowedHeaders(Arrays.asList("*"));

    // Allow credentials (cookies, if needed)
    config.setAllowCredentials(true);

    return source;
}
```

Interview Q&A:

Q: Why is CORS needed? A: Browsers block requests from one origin (e.g., `localhost:5173`) to another (e.g., `localhost:8080`) by default for security. CORS configuration allows these cross-origin requests.

🔑 Complete Login-to-API-Call Flow

```
┌──────────────────────────────────────────────────────────────┐
│  STEP 1: User Login                                          │
│  POST /api/auth/login                                        │
│  {email, password, role}                                     │
│                                                              │
│  → AuthController validates credentials                      │
│  → Generates JWT token                                       │
│  → Returns: {token, email, name, role}                       │
└──────────────────────────────────────────────────────────────┘
                    │
                    │
                    ▼
┌──────────────────────────────────────────────────────────────┐
│  STEP 2: Frontend stores token                               │
│  localStorage.setItem('token', token)                        │
└──────────────────────────────────────────────────────────────┘
                    │
                    │
                    ▼
┌──────────────────────────────────────────────────────────────┐
│  STEP 3: User navigates to protected page                    │
│  e.g., Owner Dashboard → Fetch properties                    │
│                                                              │
│  GET /api/owner/properties                                   │
│  Headers: {                                                  │
│    Authorization: "Bearer eyJhbGc..."                        │
│  }                                                           │
└──────────────────────────────────────────────────────────────┘
                    │
                    │
```

```
                    ▼
┌──────────────────────────────────────────────────────────┐
│  STEP 4: Request hits JwtAuthenticationFilter             │
│                                                           │
│  1. Extracts token from Authorization header              │
│  2. Validates token (signature + expiration)              │
│  3. Extracts claims (email, role, userId)                 │
│  4. Creates Authentication object                         │
│  5. Sets in SecurityContextHolder                         │
│  6. Sets request attributes (email, ownerId)              │
└──────────────────────────────────────────────────────────┘
                    │
                    │
                    ▼
┌──────────────────────────────────────────────────────────┐
│  STEP 5: AuthorizationFilter checks permissions           │
│                                                           │
│  Endpoint: /api/owner/**                                  │
│  Required: authenticated()                                │
│  User: authenticated with ROLE_OWNER                      │
│                                                           │
│  ☑ Access granted                                         │
└──────────────────────────────────────────────────────────┘
                    │
                    │
                    ▼
┌──────────────────────────────────────────────────────────┐
│  STEP 6: Controller method executes                       │
│                                                           │
│  @GetMapping("/owner/properties")                         │
│  public List<Property> getProperties(HttpServletRequest req) {│
│      Long ownerId = (Long) req.getAttribute("ownerId");   │
│      return propertyService.findByOwnerId(ownerId);       │
│  }                                                        │
└──────────────────────────────────────────────────────────┘
```

🔍 Common Interview Questions & Answers

Q1: What is the difference between authentication and authorization?

A:

- **Authentication** = Verifying WHO you are (login with email/password)
- **Authorization** = Verifying WHAT you can do (checking if you have permission for an action)

Q2: Why use JWT instead of sessions?

A:

- **Stateless:** No need to store session data on server
- **Scalable:** Can scale horizontally without session synchronization
- **Mobile-friendly:** Works well with mobile apps and SPAs

- **Cross-domain:** Can work across multiple domains

Q3: How is password security maintained?

A:

- Passwords are **never stored in plain text**
- We use **BCryptPasswordEncoder** which:
  - Adds random **salt** to each password
  - Uses computationally expensive hashing (slow brute-force attacks)
  - Generates different hashes for same password (due to random salt)

Q4: What happens if JWT token is stolen?

A:

- Attacker can make requests until token expires
- **Mitigations:**
  1. Short expiration time (e.g., 24 hours)
  2. HTTPS only (prevents interception)
  3. Secure storage (not in localStorage ideally, but httpOnly cookies)
  4. Token refresh mechanism (optional enhancement)

Q5: How does the filter chain order work?

A:

```
1. CORS Filter (handle OPTIONS requests)
2. JWT Authentication Filter (our custom filter)
3. Exception Translation Filter (converts security exceptions to 401/403)
4. Authorization Filter (checks permissions)
```

Order matters because JWT must be validated BEFORE authorization checks.

Q6: What is SecurityContextHolder?

A:

- **ThreadLocal storage** for authentication info
- Each request thread has its own isolated security context
- Automatically cleared after request completes
- Thread-safe for concurrent requests

📝 Summary for Interviews

Key Concepts to Remember:

1. **Spring Security uses a Filter Chain** to intercept requests before they reach controllers
2. **JWT tokens** are stateless, signed tokens containing user credentials
3. **JwtAuthenticationFilter** validates tokens and sets authentication context
4. **SecurityConfig** defines which endpoints are public vs. protected
5. **BCrypt** hashes passwords with random salt for security
6. **CORS** must be configured to allow cross-origin requests from frontend
7. **STATELESS session** policy means no server-side session storage
8. **Authorization** is role-based (CLIENT, OWNER, ADMIN)

What makes our implementation production-ready:

- ☑ Secure password hashing (BCrypt)
- ☑ JWT expiration handling
- ☑ Proper exception handling (401, 403)
- ☑ CORS configuration for frontend integration
- ☑ Role-based access control
- ☑ Stateless architecture (scalable)

End of Security Theory Document